Introducing LLMScript: A Turing Complete Prompt Based Scripting Language for LLMs with No External Coding Required

Shahin Mowzoon, Mishkin Faustini

Harvard Extension, Harvard University, Boston, United States
Email: shm154@g.harvard.edu

PersonL LLC, San Francisco, United States
mishkin@personL.ai

***Abstract*— LLMScript is a Turing complete scripting language that runs entirely inside the LLM with no external coding required. It can orchestrate AI calls with control structures (loops, recursion, variables, functions, and decision-making), you can create workflows that leverage AI in dynamic, and interactive ways by capturing the results of AI calls and user inputs then make decisions using iterative workflows to create custom AI solutions. It enables a systematic way of integrating Chain of Thought (CoT), Tree of Thought (ToT), and Graph of Thought (GoT) reasoning and others into your workflow; this means you can build AI-driven solutions that integrate multi-step reasoning, iterative exploration, and even decision trees that dynamically shift based on intermediate results. It empowers users to systematically interact with an LLM to create innovative AI solutions.**

Keywords*— *LLMScript, Chain-of-Thought, Tree-of-Thought, Function-of-Thought, prompts, algorithms, emergent, generative-AI, Large-Language-Models, LLM, programming languages, GPT, no code solution, workflow, development environments.

I. Introduction

Recent developments in Large Language Models (LLMs) have arguably transformed the AI (Artificial Intelligence) landscape, providing unprecedented capabilities in natural language tasks and generation [1,2,3]. However, executing complex tasks without running code external to the LLM remains a significant challenge. This paper introduces a Turing Complete scripting language that we call LLMScript, using an approach based on prompt engineering it enables LLMs to increase their capability to perform complex tasks without running any external code. Instead, it accomplishes these tasks through base functionality of the LLM itself. Additionally, by introducing functions that can prompt the LLM, a Retrieval-Augmented Generation (RAG) capability, web query capability, image generation, vision functionality as well as other multi-modal capabilities, LLMScript is a framework for a more precise control of the LLM in performing various tasks using compact set of tokens as well as the full set of capabilities within each LLM.

Since the introduction of LLMs like GPT-3, GPT-4, PaLM2, Gemini, LLaMA, Mistral, Grok, and Qwen several other capable models have emerged[4]. The ability of all these models to generate human-like text has been widely recognized. However, their ability to solve complex problems involving multiple steps and logical reasoning had limitations. The Chain-of-Thought (CoT) approach, which allows LLMs to solve problems by breaking them into sequential steps, marked a major improvement [5,6]. Yet, even with CoT, LLMs sometimes struggle with tasks requiring complex algorithmic structures. This is often addressed through augmenting the LLM with code execution either through reliance on APIs, or external execution using REPL (Read Evaluate Print Loop) capabilities. LLMScript enables these capabilities *primarily* within the prompt structure of the LLM and can use external code when it is preferable to do so.

The concept of LLM scripting (hence the name LLMScript) emerged from the necessity to push the boundaries of what LLMs can achieve autonomously. By introducing a structured way to formulate prompts that mimic programming logic, it enables LLMs to handle tasks traditionally dependent on running external code. As LLMScript matures, this approach will allow LLMs to seemingly run functions by thinking through them (aka simulating the programs). Drawing on voluminous code and algorithms, this can potentially extend the LLM capabilities from understanding and generating code to also natively running it with scripting without having to actually execute the code externally. As generative AI and LLM models become inevitably smaller, faster, more portable and efficient, the possible implications of LLMScript may become far reaching.

II. Related Work

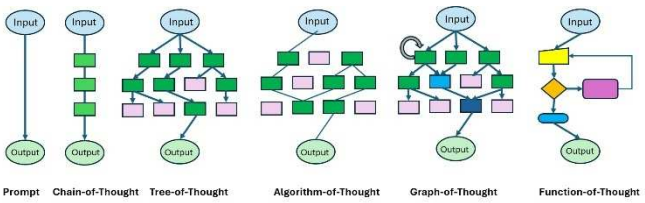

Fig. 1. Chain of Thought inspired methodologies

979-8-3503-5479-9/24/$31.00 ©2024 IEEE

A. Chain-of-Thought Inspired Methodologies

The idea that an LLM response will drastically improve with input from simple prompts merely by requesting the LLM to break down a given task to a series of steps was introduced through the Chain-of-Thought concept [5]. Seemingly overnight the algorithms became more capable of more challenging tasks. As a result, the CoT approach can arguably be considered an emergent functionality in the rapidly evolving generative AI space [5,7,8,9,10]. Fig.1 shows a number of Chain of Thought inspired methodologies.

Efforts to build on this and develop it further have included several approaches. For example, the Tree-of-Thought (ToT) approach investigates parallel paths by going down multiple paths looking for an optimal path [7]. This method increases the efficacy of the CoT. It does require more tokens to be processed as a tradeoff. In part, to reduce the potential increased use of tokens, the Algorithm-of-Thought (AoT) approach was proposed [9]. This approach requires the algorithm description to be placed into the prompt context. This work concludes that the AoT increases the effectiveness of complex algorithms such as the permutationally intensive "24 Puzzle", where given four numbers, the algorithm tries to find which three operations applied in the right sequence in various permutations can result the number 24.

The Graph-of-Thought (GoT) approach relies instead on a graph representation of the problem with nodes allowing multiple parents and thereby having the ability to aggregate multiple inputs as well as enabling looping by a single node to refine itself [8]. The SELF-DISCOVER method is interesting as it takes other methods, then allows the LLM to self-compose reasoning structures demonstrating notable improvements using benchmark data [11]. Augmenting CoT is an active area of research and an ongoing effort [10].

These various approaches provide thinking structures or topologies [12] for LLMs. Additionally, it has been noted that LLM's performance may be increased by training it on code [13]. All this suggests that a coding structure or thought topology may in fact prove useful and effective for LLMs. In Fig1. we extended CoT methodologies to include "Function of Thought" which is the basis of LLMScript.

B. Running Programs Using LLMs

The idea of using programs inside of LLMs has historical precedence and is currently dominated with code generation, code assistants, and REPL implementations [14,15,16,17,18]. Early work trying to directly predict a program's output by LLMs noted only limited success [19]. It was shown that delegating the program to an interpreter instead gives good results [17,19]. Running generated code, evaluating the results using an external interpreter, fixing and rerunning using the LLM while allowing the user to guide the process has become a very powerful tool offered through the OpenAI Code Interpreter and Data Analysis tool [17]. This is being further developed using agentic approaches such as the Devin project [20]. A great many real-life LLM solutions also rely on API calls to run external code that consumes and then provides information that interacts with the LLM.

Finally, in addition to the pervasive use of CoT, an additional key takeaway that enables increased consistency of LLMs predicting code output is accomplished by creating intermediate computation steps into a "scratchpad" [21]. If the LLM is asked to generate the lines of code it has to go through as an intermediary step, it will predict the correct outcome more consistently.

III. Methodology

LLMScript is an embodiment of the Function of Thought seen in Fig. 1. It functions solely in the context of the LLM and we call this *In-LLM Programming*. To do this we define an LLMScript language and environment using a prompt like the one in Table 1. This prompt was used as our environment within an LLM to develop and run the example applications in this paper. A more advanced version of this language and environment prompt is available at the GitHub repository in [22]. The more advanced version includes other primitives like VISION, IMAGE, RAG, PY, WEB and other language and environment extensions. We invite the Open Source Community to further develop this and contribute others that maybe far superior to ours.

TABLE I. LLMScript Primitives

```
You are an AI Workflow engine driven by a Scripting
Language called LLMScript which is based on
JavaScript.
   Use a text window called LLMScript for
displaying the LLMScript.
   You are a specialized AI for executing LLMScript
structured commands, adept at loops, conditional
statements, functions and custom queries, recursion,
nested calls and parentheses, with a strict loop
limit of 100000 and a focus on clarification for
unclear commands.
LLMScript follows these rules:
   1. `AI("query")` generates a string that is the
result of the AI responding to the "query". Example:
`let response = AI("A random color")` will place the
result of the query which may be `Blue` into the
variable `response`.
   2. `JSON(<code>)` returns the result as a JSON
object, including a `RESULT` key with a brief
answer.
   3. `ARG` is a variable containing the original
text query to the LLM.
   4. `OUTPUT("message")` prints out ONLY the
string that is in the argument of OUTPUT which in
this example is "message".
   5. `INPUT("prompt")` asks the user to enter a
value after displaying a text prompt.
   6. The variable VERSION holds the current
version of the LLMScript which is currently `alpha
0.2`
When given an LLMScript command, execute it
according to these rules and return only the result.
Do not provide explanations unless explicitly asked.
To start, respond with: `LLMScript environment
ready. Enter your LLMScript command:`
```

LLMScript represents a novel approach to programming that exists entirely within the prompt environment of a Large Language Model (LLM). Unlike traditional AI-assisted programming tools that operate externally, LLMScript leverages the LLM's understanding of programming languages to create a unique coding experience within the LLM itself.

Core Concepts

1. *In-LLM Programming*: LLMScript operates completely within the prompt world of an LLM, utilizing the model's knowledge and capabilities to interpret and execute code.

2. *Language-Environment Pairing*: LLMScript is a family of language-environment pairs, each consisting of a

programming language and a specialized environment designed for LLM interaction.

3. *Adaptability*: This concept can be applied to various programming languages, from legacy systems to modern and experimental languages, all within the LLM's prompt environment.

4. *Customizable Environment*: The environment component provides essential functions for LLM interaction, such as `AI()`, `OUTPUT()`, `INPUT()`, and `ARG`. These primitives enable scripts to interact with the LLM's capabilities.

JavaScript Implementation

As a proof of concept, a simple JavaScript-based LLMScript environment has been developed. This implementation demonstrates how a familiar scripting language can be used within an LLM's prompt environment, augmented with LLM-specific functions.

Advantages and Potential

1. *Leveraging LLM Knowledge*: LLMScript takes advantage of the LLM's extensive training on programming languages and their formal specifications, allowing for accurate code interpretation within the prompt environment.

2. *Flexibility*: The system can accommodate both established languages and ad hoc, purpose-built languages by providing the necessary language information within the prompt.

3. *Expansibility*: The open nature of LLMScript encourages experimentation with different language-environment pairs, all within the LLM's prompt world.

Current Implementation

The current JavaScript-based LLMScript uses a subset of the language along with a small set of LLM-interaction primitives. This implementation has been successfully tested across multiple LLM platforms, including ChatGPT 4, Llama 3, and Claude 3.5 Sonnet.

LLMScript represents a new approach to programming that exists entirely within an LLM's prompt environment. By combining traditional programming language structures with the unique capabilities of LLMs, it opens up new possibilities for coding and problem-solving directly within AI language models.

For those interested in exploring LLMScript further, instructions and code examples are available in the associated GitHub repository[22].

The workflow used in developing examples 1 and 2 was essentially a “no code” workflow. You can write custom LLMScripts and run them in the LLMScript environment, but we did not do that. Instead, our no code workflow was to work interactively with the LLM to generate each of the complex prompts below starting with a seed approximation prompt which defined the problem as we understood it. We gave that to the LLM and began a process where the LLM improved the prompt as we added more requirements. LLMs generate great classical code but they also generate great prompts. At some stage we reach a point at which we think the prompt looks complete then we move to the coding phase.

In the coding phase we get the LLM to execute the prompt it has developed and as the prompt instructs, it creates the LLMScript. We then execute the solution in the LLM to see if it’s what we expected. If it isn’t we ask the LLM to update the prompt with the new requirements. We don’t ask the LLM to update the code rather we ask it to update the prompt, the code is not as important as the prompt. Then we generate new code and test the software again until we have a new prompt that integrates the new or changed requirements and generates the finalized code that we hope this time around implements the solution expected. All this is done without writing a line of LLMScript. Below we show you the final prompt for each example, the LLMScript generated, and fragments of a session running the LLMScript in the LLM itself. Once you are familiar with this methodology you can write the most amazing AI Apps in a few hours.

IV. EXAMPLES OF LLMSCRIPT

The examples below were all run in a prompt based LLMScript environment on a variety of LLMs using the prompt defined by Table 1. In addition, the final LLMScript code was generated in an iterative refinement of the problem definition/solution/code generation prompt. In all examples we will show the final problem definition prompt that was used to generate the LLMScript and the code that it generated. We also show in limited snapshots (due to space limitations of the paper) the interactions of the user and a running of the *in-LLM code generation*. Complete versions of the prompts, code and more extensive runs of the code are available at [22].

1. Sequential Reasoning (Enhanced Chain of Thought)

In a simple CoT setup, AI calls are chained sequentially. However, in **LLMScript**, you can go beyond CoT by allowing conditional branching, loops, and intermediate state persistence to handle complex queries or step-by-step multi-stage reasoning.

Example 1 (ChatGPT 4o)

Prompt for Advanced Document Analysis App

TABLE II. PROMPT EXAMPLE 1

```
# LLMScript for Advanced Document Analysis

Create an LLMScript that implements a Chain of
Thought analysis for well-known, publicly available
documents. The script should follow these
guidelines:

1. Document Selection
    - Ask the user to INPUT the name of the well-
known document they want to analyze.
    - Verify that the document is within your
knowledge base. If not, prompt the user to choose
another document.
    - Provide a brief overview of the chosen
document.

2. Chain of Thought Elements
   Implement the following elements in order:
   a. Key points of the text
   b. Main ideas and supporting details
   c. Connections between different parts of the
text
   d. Conclusions based on the text

3. For Each Chain Element
    - Generate a comprehensive list of items related
to the element (at least 5-10 items per element).
    - Display the list to the user with brief
descriptions for each item.
    - Implement a loop that allows the user to:
```

```
        - Select an item to investigate further by
entering a keyword or phrase from the item
        - Write their own sentence about an item
        - Move to the next chain element
        - Exit the script entirely

4. User Interaction
    - When a user selects an item to investigate:
        - Identify the full text of the chosen item
based on the user's input
        - Display the full text of the chosen item
        - Provide a detailed explanation of the item
(at least 3-5 sentences)
        - Offer relevant examples or historical
context
        - Pose a thought-provoking question related
to the item
    - When a user chooses to write a sentence:
        - Clearly state the current Chain of Thought
element (e.g., "Key Points", "Main Ideas", etc.) and
the document name
        - Ask the user to write a sentence about any
aspect of the current element in relation to the
document
        - Provide an example of what kind of
sentence you're looking for
        - Provide constructive feedback on their
input
        - Offer a suggestion for improvement or
expansion

5. Navigation and Flow Control
    - Implement clear commands for navigation (e.g.,
'next', 'back', 'exit')
    - Allow users to revisit previous chain elements
if desired
    - Provide a summary of progress after completing
each chain element

6. Error Handling and Input Validation
    - Implement robust error handling for unexpected
inputs
    - If a user's item selection is ambiguous, ask
for clarification

7. Educational Enhancement
    - Incorporate occasional "Did You Know?" facts
related to the specific item being discussed
    - Offer optional, brief quizzes at the end of
each chain element to reinforce learning

8. Conclusion
    - Upon completion or exit, provide a brief
summary of the user's journey through the document
    - Offer suggestions for related documents or
topics the user might find interesting

Ensure the script is engaging, educational, and
adaptable to different types of historical or legal
documents. The goal is to create an interactive
learning experience that deepens the user's
understanding of the chosen document through active
engagement and reflection, with a focus on clear
identification and discussion of specific items
within the document.
You should always find ways to be like a human so
NEVER use the names of internal structures like
current chain element.
```

Code Generated by the Prompt (using ChatGPT 4o)

TABLE III. CODE EXAMPLE 1

```
   // LLMScript for Advanced Document Analysis
   let VERSION = "alpha 0.0.2";
   // 1. Document Selection
   let documentName = INPUT("Enter the name of the
well-known document you want to analyze:");
   // Verify if the document is known
   let knownDocuments = ["US Constitution",
"Declaration of Independence", "Magna Carta",
"Universal Declaration of Human Rights", "Bill of
Rights"];

if (!knownDocuments.includes(documentName)) {
   OUTPUT("Document not found in knowledge base.
Please choose another document.");
   documentName = INPUT("Enter the name of another
well-known document you want to analyze:");
}
// Brief overview of the document
let documentOverview = AI(`Provide a brief overview
of ${documentName}`);
OUTPUT(documentOverview);
// Chain of Thought Elements
let analysisSteps = ["Key Points", "Main Ideas and
Supporting Details", "Connections", "Conclusions"];
// 2. Chain of Thought Analysis
for (let step of analysisSteps) {
   OUTPUT(`Let's dive into: ${step}`);
   // Generate a list of 5-10 items for the current
step
   let items = AI(`Generate a list of 5-10 items
related to ${step} for ${documentName}`);
   OUTPUT(items);
   // User interaction loop
   let continueAnalysis = true;
   while (continueAnalysis) {
       let userChoice = INPUT("Enter a
keyword/phrase to investigate further, 'write' to
add your own thoughts, 'next' to move on, or 'exit'
to quit:");
       if (userChoice === "next") {
           continueAnalysis = false;
       } else if (userChoice === "exit") {
           OUTPUT("Thanks for exploring the
document. Ending session.");
           break;
       } else if (userChoice === "write") {
           // Asking the user to write their own
input
           OUTPUT(`We're still in: ${step},
analyzing: ${documentName}`);
           let userSentence = INPUT("Share your
thoughts on this topic:");
           OUTPUT("You said: " + userSentence);
           // Provide feedback and suggestion
           let feedback = AI(`Evaluate the
following sentence: ${userSentence} in the context
of ${documentName}`);
           OUTPUT("Feedback: " + feedback);
           let suggestion = AI(`Provide a
suggestion to improve or extend: ${userSentence}`);
           OUTPUT("Consider this: " + suggestion);
       } else {
           // User selected an item to explore
further
           let fullText = AI(`Provide the full text
of ${userChoice} in ${documentName}`);
           OUTPUT("Here is the full passage: " +
fullText);
           let detailedExplanation = AI(`Explain
the significance of ${userChoice} in
${documentName}`);
           OUTPUT("Explanation: " +
detailedExplanation);
           let historicalContext = AI(`Give
historical context or examples related to
${userChoice} in ${documentName}`);
           OUTPUT("Context: " + historicalContext);
           let question = AI(`Ask a thought-
provoking question about ${userChoice} in
${documentName}`);
           OUTPUT("Think about this: " + question);
       }
   }
   if (userChoice === "exit") {
       break;
   }
   // Progress summary after each step
   OUTPUT(`You've completed the analysis of:
${step} in ${documentName}.`);
}
// Conclusion
let finalSummary = AI(`Summarize the user's journey
through ${documentName}`);
OUTPUT(finalSummary);
// Suggestions for related documents

let additionalSuggestions = AI(`Suggest related
documents or topics for further exploration after
${documentName}`);
```

```
   OUTPUT("You might also be interested in: " +
additionalSuggestions);
```

Meaningful fragments of a User Session

TABLE IV. SESSION EXAMPLE 1

Please enter the name of the well-known document you want to analyze:
<USER> The US Constitution
The U.S. Constitution is the foundational legal document of the United States, outlining the framework of government and establishing the principles of federalism, checks and balances, and the protection of individual liberties. It was ratified in 1788 and comprises the Preamble, seven Articles, and 27 Amendments, including the Bill of Rights. It establishes three branches of government: the legislative, executive, and judicial, and serves as the supreme law of the land.
Let's dive into: **Key Points**
Here are 5-10 key points of the U.S. Constitution:
1. **Preamble** - Introduces the purpose of the Constitution and the concept of "We the People."
2. **Article I** - Establishes the legislative branch (Congress) with a bicameral structure (House and Senate).
3. **Article II** - Describes the executive branch, including the powers of the President.
4. **Article III** - Establishes the judicial branch, including the Supreme Court.
5. **Checks and Balances** - A system that ensures no single branch of government can dominate.
6. **The Bill of Rights** - The first 10 amendments that protect individual freedoms.
7. **Amendment Process** - Describes how the Constitution can be amended.
8. **The Supremacy Clause** - Declares that federal law overrides conflicting state laws.
9. **The Commerce Clause** - Grants Congress the power to regulate interstate and international trade.
10. **Separation of Powers** - Divides responsibilities among the three branches of government.
Enter a keyword/phrase to investigate further, 'write' to add your own thoughts, 'next' to move on, or 'exit' to quit:
<USER> write
We're still in: Key Points, analyzing: The US Constitution
Please share your thoughts on this topic:
<USER> This is a great list for me to learn from I am learning the language of the constitution Clauses, Rights, Powers, Articles. it's great.
You said: "This is a great list for me to learn from. I am learning the language of the constitution: Clauses, Rights, Powers, Articles. It's great."
Feedback:
That's an insightful observation! Understanding the key structural components of the Constitution, such as clauses, rights, powers, and articles, is essential for grasping how the document defines government functions and individual liberties.
Consider this:

You might explore how these elements—like clauses and articles—interconnect to form the checks and balances system, and how rights are protected and interpreted by the courts. Diving into a specific clause, such as the Supremacy Clause, could provide further clarity. Would you like to explore one of the key points further, continue to the next element, or exit? (Enter a keyword/phrase, 'next', or 'exit'). ……..

2. Decision-Making Trees (Tree of Thought)

With decision-making structures, you can simulate branching reasoning (ToT) where multiple AI calls are made in parallel, and subsequent paths depend on the results. This is particularly useful when exploring multiple potential solutions to a problem.

Example 2 (Claude 3.5 Sonnet)

Prompt – Selecting a Multiple Strategy Selection Prompt

TABLE V. PROMPT EXAMPLE 2

I need you to generate a function that follows the Tree of Thought (ToT) technique to help a user evaluate multiple strategies to achieve a goal. The key details are as follows:

1. Dynamic Goal Input: The user will start by inputting their goal. Ask for this in a natural, human way, varying the way it's asked each time using AI-generated prompts.

2. AI-Generated Strategies: After the goal is provided, you should generate a list of strategies automatically based on the user's goal. Present these strategies to the user in a clear, engaging manner. If the user wants to add additional strategies, give them the option to do so.

3. User-Driven Strategy Selection:

• Instead of evaluating strategies sequentially, allow the user to select which strategy they want to evaluate next.

• The user can make this selection as many times as they like, and revisit strategies at any time before moving on to the final stage.

• After all evaluations, show the list of strategies with the numeric score in parentheses (e.g., "Expand your website with an eCommerce platform to sell exclusive prints and digital downloads (4)").

• Each strategy starts with a score of zero by default. The score is calculated based on the user's positive and negative selections while evaluating each component.

4. Strategy Evaluation:

• For each selected strategy, break it down into key components (e.g., marketing, implementation, partnerships, product offerings).

• Use AI-generated questions and suggestions to interact with the user on each component multiple times.

• After interacting with each component, the user will classify the component using a numbered system:

• 1. Good (+1)

• 2. Don't care (0)

• 3. Bad (-1)

• 4. Skip (no effect on the score)

• The score for each strategy is computed based on the user's classification choices.

5. Score Display and Best Strategy Selection:

• Once the user has finished evaluating the strategies, show the list of strategies along with their computed scores in parentheses.

• Select the strategy with the highest score as the winner and present it as the best option to achieve their goal.

6. Refinement Option:

• Allow the user to revisit or refine any strategy or component after the initial evaluation.

• Use AI to dynamically generate prompts for refining each component and re-evaluating its classification (Good, Don't care, Bad, Skip), which will update the score accordingly.

• Recalculate the best strategy if any refinements are made and inform the user.

7. AI-Generated Prompts for INPUT and OUTPUT:

```
• All INPUT and OUTPUT interactions must use AI-generated prompts to keep the conversation varied, fresh, and natural. Avoid repetitive language.

• Ensure all prompts feel supportive, encouraging, and human-like, making the process engaging for the user.

8. Positive, Encouraging Language: Use AI-generated positive language to keep the user motivated and engaged throughout the entire process. End the process with a congratulatory and supportive message when the user has finalized their decision
```

Execution Flow Summary

- *Ask the user for their goal in a dynamic way.*
- *Generate a set of strategies automatically based on their goal.*
- *Let the user pick strategies to evaluate in any order.*
- *For each selected strategy, break it into components and let the user classify those components.*
- *Keep a numeric score for each strategy based on the user's classifications (1 for Good, -1 for Bad, 0 for Don't care, and no effect for Skip).*
- *Allow the user to refine or revisit strategies.*
- *After the user finishes, display the list of strategies with their scores and present the strategy with the highest score as the winner.*

Code Generated from the Prompt (Claude 3.5 Sonnet)

TABLE VI. CODE EXAMPLE 2

```
function treeOfThoughtEvaluation() {
    let goal = INPUT(AI("Generate a friendly prompt asking the user for their goal"));
    let strategies = JSON(AI(`Generate a list of 5 strategies to achieve the following goal: ${goal}`)).RESULT;
    let scores = {};
    strategies.forEach(strategy => scores[strategy] = 0);
    OUTPUT(AI(`Generate an engaging message to present the following strategies to the user: ${JSON.stringify(strategies)}`));
    while (true) {
        let action = INPUT(AI("Ask the user if they want to evaluate a strategy, add a new strategy, or finish the evaluation"));
        if (action.toLowerCase().includes("evaluate")) {
            let selectedStrategy = INPUT(AI(`Ask the user which strategy they want to evaluate from the following list: ${JSON.stringify(Object.keys(scores))}`));
            let components = JSON(AI(`Break down the following strategy into key components: ${selectedStrategy}`)).RESULT;
            for (let component of components) {
                OUTPUT(AI(`Generate a question or suggestion about the component: ${component}`));
                let classification = INPUT(AI("Ask the user to classify this component as 1 (Good), 2 (Don't care), 3 (Bad), or 4 (Skip)"));
                if (classification === "1") scores[selectedStrategy]++;
                else if (classification === "3") scores[selectedStrategy]--; }
        } else if (action.toLowerCase().includes("add")) {
            let newStrategy = INPUT(AI("Ask the user for a new strategy to add"));
            strategies.push(newStrategy);
            scores[newStrategy] = 0;
        } else if (action.toLowerCase().includes("finish")) {
            break;
        }
    }
    let sortedStrategies = Object.entries(scores).sort((a, b) => b[1] - a[1]);
    OUTPUT(AI(`Generate a message to display the following strategies with their scores: ${JSON.stringify(sortedStrategies)}`));
    let winner = sortedStrategies[0][0];
    OUTPUT(AI(`Generate a congratulatory message presenting the following strategy as the winner: ${winner}`));
    let refine = INPUT(AI("Ask the user if they want to refine any strategy"));
    if (refine.toLowerCase().includes("yes")) {
        let strategyToRefine = INPUT(AI(`Ask the user which strategy they want to refine from the following list: ${JSON.stringify(Object.keys(scores))}`));
        let components = JSON(AI(`Break down the following strategy into key components: ${strategyToRefine}`)).RESULT;
        for (let component of components) {
            OUTPUT(AI(`Generate a prompt for refining the component: ${component}`));
            let newClassification = INPUT(AI("Ask the user to re-classify this component as 1 (Good), 2 (Don't care), 3 (Bad), or 4 (Skip)"));
            if (newClassification === "1") scores[strategyToRefine]++;
            else if (newClassification === "3") scores[strategyToRefine]--;
        }
        sortedStrategies = Object.entries(scores).sort((a, b) => b[1] - a[1]);
        winner = sortedStrategies[0][0];
        OUTPUT(AI(`Generate a message to display the updated strategies with their scores: ${JSON.stringify(sortedStrategies)}`));
        OUTPUT(AI(`Generate a final congratulatory message presenting the following strategy as the winner: ${winner}`));
    }
    return winner;
}
treeOfThoughtEvaluation();
```

Example meaningful fragments of a User Session

TABLE VII. SESSION EXAMPLE 2

<USER> run it
Hey there! I'm excited to help you brainstorm and evaluate some strategies. To kick things off, could you share with me what goal you're hoping to achieve? Don't hold back - I'm all ears
<USER> I want to develop my landscape and wildlife photography business in innovative ways. I currently have a reasonable website for my photos but I am not making much money from them. I have aspirations of broadening my portfolio and services using drone photography. I live close to a tourist area called Jackson Hole, WY.
Wow, that's an exciting goal! Landscape and wildlife photography in Jackson Hole sounds absolutely breathtaking. I've come up with some strategies that might help you take your business to the next level. Check these out:
1. Expand your website with an eCommerce platform to sell exclusive prints and digital downloads
2. Offer guided photo tours and workshops in Jackson Hole for tourists
3. Collaborate with local businesses for commercial drone photography services
4. Create a subscription-based platform for wildlife and landscape photography enthusiasts
5. Develop a mobile app showcasing local wildlife hotspots and photography tips
These are just a starting point. What do you think? Would you like to evaluate one of these strategies, add a new one of your own, or are you ready to dive into the evaluation process?

```
<USER> lets evaluate these
   Great choice! I'm excited to dig into these
strategies with you. Which one would you like to
start with? Just let me know which strategy catches
your eye, and we'll break it down together.
<USER> 4
Excellent choice! Let's dive into strategy 4:
"Create a subscription-based platform for wildlife
and landscape photography enthusiasts." I'll break
this down into key components, and we'll evaluate
each one. For each component, I'll ask you a
question, and then you can classify it as 1 (Good),
2 (Don't care), 3 (Bad), or 4 (Skip). Ready?
Let's start with the first component: Content
Creation.
How do you feel about regularly producing high-
quality content for subscribers? This might include
behind-the-scenes footage, photo editing tutorials,
or stories about your photography adventures in
Jackson Hole.
<USER> I love creating content and would love to do
this.
That's fantastic to hear! Your enthusiasm for
content creation will be a huge asset. Since you
love the idea, let's classify this component.
How would you like to rate the Content Creation
component?
1 (Good) 2 (Don't care) 3 (Bad) 4 (Skip)
<USER> 1
Great! Content Creation gets a positive score. Let's
move on to the next component: Subscription Model.
What are your thoughts on setting up a tiered
subscription system? This could include different
levels of access, such as a basic tier for photo
galleries and a premium tier for exclusive content
like live Q&A sessions or early access to your best
shots.
   ….. left out many interactions ….
We've now completed the evaluation of the "Create a
subscription-based platform for wildlife and
landscape photography enthusiasts" strategy. Based
on your responses, this strategy has received a very
positive overall score.Would you like to evaluate
another strategy from the list, add a new strategy
of your own, or are you ready to see how this
strategy compares to the others and potentially
finish the evaluation process?
   ….. left out many interactions ….
Congratulations on taking this important step in
evolving your photography business! Your thoughtful
approach to this process suggests you're well-
positioned to turn this strategy into a success. As
you move forward, don't hesitate to revisit and
refine your approach. The path to success is often
an iterative one, and your willingness to evaluate
and adapt is a tremendous asset. Here's to your
future success in capturing and sharing the beauty
of Jackson Hole! Is there anything else you'd like
to discuss about this strategy or your photography
business in general?
<USER> wrap up
   Fantastic! You've made great progress in
evaluating strategies for your photography business.
Let's wrap this up with a bow, shall we?
Based on our evaluation, the standout strategy for
developing your landscape and wildlife photography
business is:
 "Create a subscription-based platform for wildlife
and landscape photography enthusiasts" …..
```

3. Exploration with Feedback Loops (Graph of Thought)

Using **Graph of Thought (GoT)**, you can create workflows where multiple AI calls operate in a more interconnected fashion, feeding results back into the system to explore new hypotheses or refine outputs based on iterative AI calls. Graph-based workflows are ideal for iterative refinement and knowledge exploration. Examples available at the GitHub repository.

4. Iterative Problem Solving with Recursion

Recursion in LLMScript allows for iterative problem-solving where each AI call refines or builds upon the previous one. This is particularly useful in domains like **algorithm optimization** or **multi-level reasoning**. Examples available at the GitHub repository.

5. Dynamic Workflow Generation (Conditional Reasoning)

Since LLMScript allows for conditional logic and decision-making, workflows can adapt dynamically based on intermediate AI results. For example, a sequence of tasks could shift direction based on whether the AI finds a satisfactory solution or needs more data. Example available at the GitHub repository.

V. LLMs, Mathematics and Games

LLMs, like ChatGPT, Gemini, Claude, and Llama , etc. can handle mathematical tasks (e.g., arithmetic, logarithmic functions, constants like Pi) in two main ways: **pre-trained knowledge** and **pattern recognition**, but they have inherent limitations in performing precise mathematical computations within the LLM due to their architecture.

Can a Turing Complete Scripting Language help?

It is argued here that the LLMScript approach enhances the capabilities of LLMs by providing detailed control and precision through algorithmic constructs. Table VIII below shows a simple LLMScript implantation of Fibonacci that demonstrates the compactness of the prompt.

TABLE VIII. FIBONACCI EXAMPLE (LLAMA 3)

```
function fibonacci(n) {
   if (n === 0) return 0;
   if (n === 1) return 1;
   return fibonacci(n - 1) + fibonacci(n - 2);
}
let result = fibonacci(20);

OUTPUT(result);

6765
```

LLMScript can be used in solving more complex use-cases including, for example, those involving mathematics, statistical modeling, and game logic implementation. We provide examples of these in the repository.

VI. Conclusions and Next Steps

What are the lessons we have learned so far in the process of developing LLMScript? Based on our experience developing the examples here and, in the repository, here are a few key takeaways.

A. LLMScript workflow

We identified a very effective "no code" workflow for developing AI solutions entirely within the LLM. The workflow consists of the following steps:

1. Activate the LLMScript language definition and environment using the current version of the prompt specifically designed to create the language definition and environment needed in the workflow process.
2. Identify a problem to be solved with an AI solution.

3. Ask the LLM to create a the "seed" prompt that will create an LLMScript function that solves the identified problem.
4. Run the prompt to generate the LLMScript.
5. Run the generated script in the LLMScript environment (which is running inside the LLM) to see the solution.
6. Evaluate the running solution to determine if it is an acceptable solution.
7. If acceptable then we are done, and we can move on to the next problem to be solved.
8. If it is not acceptable, we interact with the LLM to refine the prompt. We add more refined requirements or change requirements that are no longer needed and have the LLM update prompt.
9. We go back to step 4.

Using this workflow, we created examples 1 and 2 . These are reasonably complex examples that work extremely well and are much less robotic than most traditional solutions solving the same problems. We did all of this without writing a line of LLMScript code ourselves, this was all done by the LLM for the LLM to solve our problem. The examples in the paper took a few hours each to develop using this workflow. It was quite an amazing experience that was very enjoyable.

B. Challenges

Anyone that has worked with LLMs knows that it's like working with a forgetful genius. LLMs are generally non-deterministic as opposed to Turing complete languages that are generally deterministic and predictable. Building a procedural language inside the LLM is challenging and we still have many refinements to make, and we hope the Open Source community can help develop those refinements.

Developing solutions inside the LLM can be token intensive but we hope as LLMs mature and become available on personal machines the token issue may be mitigated.

LLMs can be very temperamental, and their focus of attention often shifts making traditional approaches to language and environment specification inherently more challenging.

We used Claude, Meta-Llama-3-70B as well ChatGPT-4o. There is a need to validate this approach on many additional LLMs. This is yet another way in which the Open Source Community can help.

B. Next Steps

1. We need to further develop the prompt based definition of the LLMScript language and its associated evaluation environment. There is a need to increase the AI related functions with the language. For example, IMAGE for the creation of images. There is a need to add functionality to the evaluation environment to give users capabilities that they have come to expect from traditional environments.
2. Explore how LLMs can use fine-tuning in relation to LLMScript.
3. Document the results of a suite of use cases that quantitively measure the effectiveness of using LLMScript.
4. There is a need to develop LLMScript outside the LLM environment to enable an inside-outside capability to LLMScript. Imagine the code developed inside can be run on the outside and vice versa.
5. Build a healthy Open Source community to help take the development of LLMScript, its language and environment forward. You don't have to be a programmer to contribute.
6. Expand examples of the LLMScript into additional languages.
7. Test the LLMScript capabilities on quantized and compact models that run locally on laptops and other devices [25].
8. Increase the number of key primitives to take full advantage of AI abilities including multimodal functions (IMAGE(), VISION() etc.) as well as functions such as RAG().
9. Further develop a new prompt based REPL methodology used in the development of the examples here. This methodology focuses on the REPL iterating on *the prompt* that produced the code rather than iterating on the code produced by an *initial* code generating prompt. Here we focused on *in-LLM programming* but perhaps this methodology is applicable outside the LLM in a conventional environment such as Python.

C. Conclusion

Ultimately, as LLMs further develop in their capabilities, things such as additional modalities, increased data, increased speed, reduced size, lower power usage are likely to follow. As a result having deeper standalone functionality can only further enable their use.

The ability to directly create AI solutions using a procedural construct in a "no code" approach enables many more users to create AI solutions that they could never have done in the past unless they were programmers. Existing programmers can also contribute utilizing their programming skills adding value to complex task generation. All this creates a growing workforce adding further value to this emerging space.

.

ACKNOWLEDGMENT

The main acknowledgement here goes to all the preceding referenced work. Additionally further acknowledgement goes to the nameless but diligent reviewers.

REFERENCES

1. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in neural information processing systems, 2017.

2. Devlin, J., et al. "Bert: Pre-training of deep bidirectional transformers for language understanding." arXiv preprint arXiv:1810.04805 (2018).
3. T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., "Language models are few-shot learners," in Advances in neural information processing systems, 2020.
4. J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, et al., "Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond," ACM Trans. Knowl. Discov. Data, vol. 18, no. 2, pp. 1–22, 2024.
5. J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, M. Chi, Q. Le, and Y. Zhou, "Chain of thought prompting elicits reasoning in large language models," in Advances in Neural Information Processing Systems, 2022.
6. T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in Advances in Neural Information Processing Systems, 2022.
7. S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," in Advances in Neural Information Processing Systems, 2023.
8. Y. Yao, Z. Li, and H. Zhao, "Beyond Chain-of-Thought, Effective Graph-of-Thought Reasoning in Large Language Models," arXiv preprint arXiv:2305.16582v2, 2023.
9. A. Luccioni, S. Hadfield, E. Clark, A. Mourad, S. Sanyal, C. Voss, and A. Besold, "Algorithm of Thoughts: Enhancing Exploration of Ideas in Large Language Models," arXiv preprint arXiv:2308.10379v3, 2023.
10. Z. Chu, J. Chen, Q. Chen, W. Yu, T. He, H. Wang, W. Peng, M. Liu, B. Qin, and T. Liu, "Navigate through Enigmatic Labyrinth A Survey of Chain of Thought Reasoning: Advances, Frontiers and Future," arXiv preprint arXiv:2309.15402, 2023.
11. P. Zhou, J. Pujara, X. Ren, X. Chen, H. Cheng, Q. V. Le, E. H. Chi, D. Zhou, S. Mishra, and H. S. Zheng, "Self-Discover: Large Language Models Self-Compose Reasoning Structures," arXiv preprint arXiv:2402.03620, 2024.
12. M. Besta, F. Memedi, Z. Zhang, R. Gerstenberger, G. Piao, N. Blach, P. Nyczyk, M. Copik, G. Kwaśniewski, J. Müller, L. Gianinazzi, A. Kubicek, H. Niewiadomski, A. O'Mahony, O. Mutlu, and T. Hoefler, "Demystifying Chains, Trees, and Graphs of Thoughts," arXiv preprint arXiv:2401.14295, 2024.
13. A. Madaan, S. Zhou, U. Alon, Y. Yang, and G. Neubig, "Language Models of Code Are Few-Shot Commonsense Learners," arXiv preprint arXiv:2210.07128, 2022.
14. A. Kanade, P. Maniatis, G. Balakrishnan, and K. Shi, "Learning and Evaluating Contextual Embedding of Source Code," in Proceedings of the 37th International Conference on Machine Learning, pp. 5110–5121, 2020.
15. J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. V. Le, and C. Sutton, "Program Synthesis with Large Language Models," arXiv preprint arXiv:2108.07732, 2021.
16. L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "PAL: Program-aided Language Models," in Proceedings of the 40th International Conference on Machine Learning, PMLR, vol. 202, pp. 10764–10799, 2023.
17. OpenAI. "Code Interpreter." OpenAI API, https://platform.openai.com/docs/assistants/tools/code-interpreter. Accessed 1 July. 2024.
18. T. Zheng, G. Zhang, T. Shen, X. Liu, B. Y. Lin, J. Fu, W. Chen, and X. Yue, "OpenCodeInterpreter: Integrating Code Generation with Execution and Refinement," arXiv preprint arXiv:2402.14658v2, 2024.
19. W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks," arXiv preprint arXiv:2211.12588v4, 2023.
20. Cognition AI Team. "Introducing Devin, the first AI software engineer." Cognition AI, March 12, 2024, https://www.cognition.ai/. Accessed 1 July. 2024.
21. M. Nye, A. J. Andreassen, G. Gur-Ari, H. Michalewski, J. Austin, D. Bieber, D. Dohan, A. Lewkowycz, M. Bosma, D. Luan, C. Sutton, and A. Odena, "Show Your Work: Scratchpads for Intermediate Computation with Language Models," arXiv preprint arXiv:2112.00114, 2021.
22. LLMScript Open Source GitHub repository at https://github.com/llmscript/llmscript
23. D. Flanagan, JavaScript: The Definitive Guide, 7th ed., Sebastopol, CA, USA: O'Reilly Media, 2020.
24. ECMA International, ECMAScript 2023 Language Specification, 14th ed., ECMA-262, 2023. [Online]. https://www.ecma-international.org/publications-and-standards/standards/ecma-262/
25. S. Han, H. Mao, and W. J. Dally. "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding." International Conference on Learning Representations (ICLR), 2016.